Routing Strategies for Ring Topologies in OMNeT++

Álvarez, Lis Ailín; Pinilla, Lautaro; Sabino, Lucas

Grupo 23, Facultad de Matemática, Astronomía, Física y Computación (FaMAF)
Universidad Nacional de Córdoba, Argentina

Abstract

This paper evaluates a baseline routing algorithm in an OMNeT++ ring network of 8 nodes, which forwards all packets through a fixed interface. Two traffic scenarios are analyzed: two sources and seven sources sending to node 5, with varying inter-arrival times. Metrics include delay, hop count, buffer occupancy, and link utilization. The baseline shows asymmetric load, long delays, and a stability threshold near 7 s, below which queues grow indefinitely. To overcome this, we propose a distance-based algorithm using both ring directions, discovering the network size autonomously via an initial exchange of hello and echo messages without OMNeT++ topology functions. Our algorithm selects the shortest path, balances traffic, and avoids loops. Results show significant reductions in delay and buffer usage, and a higher stability threshold (from $IAT > 7\,\text{s}$ to $IAT > 4\,\text{s}$ theoretically). The design scales to any ring size through this neighbor discovery mechanism. No routing loops were observed.

Index Terms— Network simulation, OMNeT++, routing, ring topology, performance evaluation, queueing, stability.

1 Introducción

Este artículo reporta el análisis y diseño de estrategias de enrutamiento implementadas sobre el modelo de red del Laboratorio 4. Se estudia una red en anillo de ocho nodos en la que el algoritmo baseline reenvía siempre los paquetes por la misma interfaz (`toLnk[0]`), sin aprovechar el camino alternativo disponible. El objetivo es instrumentar métricas de desempeño, incluyendo demora, cantidad de saltos, ocupación de buffers y utilización de enlaces, evaluar el comportamiento del algoritmo baseline en distintos escenarios de tráfico y diseñar una estrategia de enrutamiento mejorada. El trabajo se organiza de la siguiente manera: la Sección 2 describe el modelo de simulación; la Sección 3.1 presenta el análisis del algoritmo baseline; la Sección **??** detalla el diseño del nuevo algoritmo; la Sección **??** compara los resultados obtenidos; y finalmente la Sección 6 resume las conclusiones del trabajo.

La principal limitación del baseline es el uso exclusivo de una de las dos interfaces, lo que satura un sentido del anillo mientras el otro permanece vacío. Esto provoca demoras innecesarias, buffers largos y baja eficiencia. Una estrategia será considerada mejor si reduce la demora media, equilibra la carga entre ambas direcciones, evita bucles y permite sostener tasas de tráfico más altas antes de la saturación.

Se propone un algoritmo de enrutamiento por distancia mínima que utiliza ambos sentidos del anillo, descubriendo la topología mediante mensajes hello. De esta forma, cada paquete sigue el camino más corto al destino. Se espera una mejora significativa en demora, balance de carga y umbral de estabilidad respecto del baseline, sin introducir bucles de enrutamiento.

2 Escenario y modelo de simulación

La topología utilizada es un anillo de 8 nodos numerados del 0 al 7. Cada nodo posee dos interfaces full-duplex (denominadas `lnk[0]` y `lnk[1]`) que lo conectan con sus dos vecinos. Internamente, cada nodo se compone de tres submódulos: la capa de aplicación (`app`), que genera paquetes según una distribución exponencial con tamaño fijo; la capa de red (`net`), que implementa la lógica de enrutamiento; y dos capas de enlace (`lnk[0]` y `lnk[1]`), cada una con su propio buffer y un servidor que modela la transmisión a 1 Mbps con un retardo de propagación de 100 μs. La Figura 1 muestra la disposición de la red.

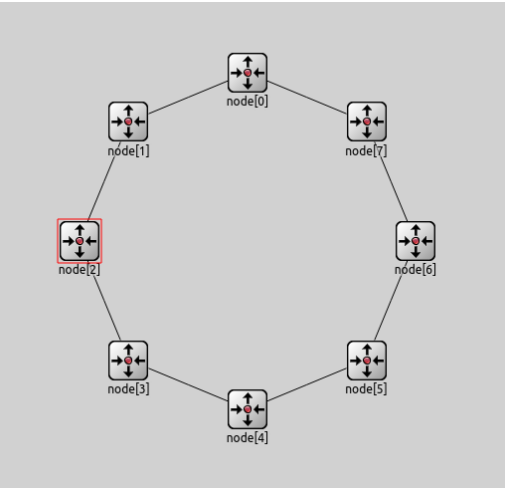


Figure 1: Topología en anillo de 8 nodos.

2.1 Topología y configuración de tráfico

Todos los enlaces son full-duplex, con capacidad de 1 Mbps y retardo de propagación de 100 μs. La capa de aplicación genera paquetes de 125000 bytes con tiempos entre llegadas exponenciales. En el **Caso 1**, solo los nodos 0 y 2 actúan como fuentes, ambos con destino en el nodo 5 y una tasa media de 1 paquete por segundo (`exponential(1)`). En el **Caso 2**, todos los nodos excepto el 5 (es decir, los nodos 0,1,2,3,4,6,7) generan tráfico hacia el nodo 5, y se varía el valor de `interArrivalTime` en el rango de 0.5 a 8 segundos (específicamente: 0.5, 1, 2, 4, 6, 8) para estudiar la estabilidad. La Tabla 1 resume los parámetros empleados.

Table 1: Configuración experimental por escenario.

Parámetro	Caso 1	Caso 2	Comp.
Fuentes activas	`n[0]`, `n[2]`	7 → `n[5]`	1 y 2
Destino	`n[5]`	`n[5]`	idem
`packetByteSize`	125000 bytes	125000 bytes	125000 bytes
`interArrivalTime`	`exponential(1)`	`exponential(0.5 a 8)`	–
T. simulación (s)	200	200	200
Semilla	1	1	1

2.2 Métricas registradas

Para evaluar el rendimiento del sistema de enrutamiento se instrumentaron las siguientes métricas en los módulos correspondientes:

- **Demora extremo a extremo (s)**: medida en la capa de aplicación del nodo destino, calculando la diferencia entre el tiempo de recepción y el de creación del paquete.
- **Cantidad de saltos por paquete**: se incrementa un contador en la capa de red de cada nodo que reenvía el paquete; el valor final se registra en la aplicación destino.
- **Ocupación de buffer (paquetes)**: se registra la longitud instantánea de la cola en cada capa de enlace cada vez que se encola o desencola un paquete.
- **Utilización del enlace (%)**: se acumula el tiempo total de transmisión de paquetes en cada interfaz y se divide por el tiempo total de simulación.

La Tabla 2 resume las métricas y sus módulos.

Table 2: Métricas instrumentadas en el simulador.

Métrica	Módulo	Unid.
Demora E2E (promedio)	`app`	s
Paquetes enviados	`app`	adim.
Paquetes recibidos	`app`	adim.
Saltos por paquete (promedio)	`app`	adim.
Saltos por paquete (máximo)	`app`	adim.
Ocupación buffer (promedio)	`lnk`	pkts
Ocupación buffer (máximo)	`lnk`	pkts
Utilización enlace	`lnk`	%

3 Tarea de análisis: evaluación del kickstarter

En esta sección se presenta la evaluación de rendimiento del algoritmo de enrutamiento base (kickstarter) implementado en la topología de anillo. Para diagnosticar sus limitaciones, el análisis se divide en dos escenarios de simulación. El Caso 1 evalúa el comportamiento y la distribución de carga de la red ante fuentes específicas (`node[0]` y `node[2]` hacia `node[5]`). Dado que el algoritmo reenvía siempre los paquetes por la interfaz `toLnk[0]`, el tráfico se ve forzado a viajar invariablemente en sentido antihorario ($2 \to 1 \to 0 \ldots \to 5$), adoptando rutas que no siempre son óptimas. Por su parte, el Caso 2 busca determinar el punto de saturación de los buffers frente a un incremento generalizado y sostenido del tráfico. Las métricas obtenidas en ambos experimentos servirán como línea base empírica para justificar el diseño de un algoritmo de enrutamiento mejorado.

3.1 Caso 1: tráfico configurado (`node[0]` y `node[2]` → `node[5]`)

En esta simulación, evaluamos el desempeño de la red bajo un escenario asimétrico, donde solo `node[0]` y `node[2]` generan tráfico hacia `node[5]`. Según los resultados resumidos en la Tabla 3, el algoritmo baseline obtiene una demora media de 53.25 s, lo cual evidencia una congestión importante. En cuanto al uso de recursos, se observó que la ocupación máxima a nivel global alcanzó los 199 paquetes y la utilización máxima de enlace llegó al 98%, demostrando que prácticamente el enlace en la dirección de transmisión elegida por el enrutamiento estático está saturado.

Al analizar el tráfico de forma individual (ver Fig. 2), se hace evidente el problema estructural del algoritmo baseline: el flujo de tráfico estático hace que el `node[0]` absorba tanto los paquetes generados por él mismo, como los que arriban desde y a través del `node[2]` (en su camino hacia el `node[5]`). Esto transforma al `node[0]` e indirectamente sus puertos `lnk[0]` en un cuello de botella crítico (con un buffer que llega al tope de 199 paquetes y en 98% utilización de enlace), mientras que los recursos propios iniciales de `node[2]` tienen menor presión.

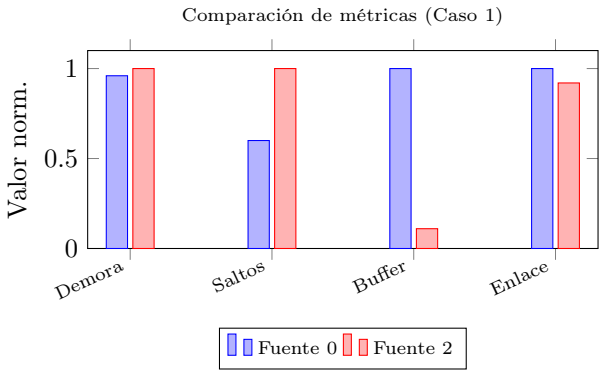


Figure 2: Métricas normalizadas para el Caso 1 (baseline). La métrica Buffer (máximo) y Enlace correlacionan los nodos `node[0]` para Fuente 0 y `node[2]` para Fuente 2.

Como consecuencia directa de esta congestión, surge una penalización generalizada e ineficiente en las demoras. Mientras que el tráfico de la Fuente 2 requiere 5 saltos para llegar a destino, la Fuente 0 solo requiere 3. Sin embargo, las demoras promedio son extremadamente similares (en torno a los 53–54 s para ambas). Esto demuestra que el tiempo extra de transmisión por dar más saltos termina siendo insignificante comparado con el masivo retraso que ambos flujos sufren (queueing delay) al quedar atascados en la misma cola del nodo saturado (`node[0]`), haciendo que los paquetes tarden prácticamente lo mismo en llegar al destino sin importar su punto de partida.

Este cuello de botella y las demoras asociadas subrayan la necesidad de implementar esquemas de ruteo basados en métricas de distancia que balanceen la carga de red por ambos sentidos del anillo (para ir por el camino más corto).

Table 3: Resultados globales del Caso 1 (baseline).

Métrica	Prom.	Máx.	Desv.
Demora (s)	53.25	–	–
Saltos	3.8	5	0.98
Ocupación buffer	21.8	199	43.4
Util. enlace (%)	29.5	98.0	45.2

3.2 Caso 2: tráfico sostenido hacia `node[5]`

En este escenario los siete nodos restantes del anillo (`node[0]`–`node[4]`, `node[6]` y `node[7]`) generan tráfico simultáneamente hacia `node[5]` con el mismo `packetByteSize` (125 000 bytes = 1 Mbit) e `interArrivalTime` exponencial. Se exploraron seis valores: 0.5, 1, 2, 4, 6 y 8 segundos, con el objetivo de determinar el umbral de estabilidad de la red bajo el algoritmo baseline.

Dirección del tráfico y cuello de botella

El archivo `.ned` define las conexiones del anillo de la siguiente manera:

```
node[0].toNod[1] <--> node[1].toNod[0]
node[1].toNod[1] <--> node[2].toNod[0]
...
node[7].toNod[1] <--> node[0].toNod[0]
```

El módulo `Net` del baseline reenvía siempre por `toLnk[0]`, que corresponde a `toNod[0]`. Dado que `toNod[0]` de cada nodo n está conectado a `toNod[1]` del nodo $n-1$, los paquetes viajan en dirección **decreciente**: $n \to n{-}1 \to \cdots \to 5$. Los nodos 6 y 7 quedan al otro lado del destino, por lo que recorren el camino largo: $6 \to 5$ (1 salto) y $7 \to 6 \to 5$ (2 saltos), mientras que el `node[4]` debe atravesar todo el anillo ($4 \to 3 \to \cdots \to 6 \to 5$, 7 saltos). La Tabla 4 resume los flujos acumulados en cada enlace.

Table 4: Flujos acumulados por enlace bajo routing baseline (dirección decreciente, destino `node[5]`).

Enlace	Flujos	Nodos que transitan
`node[4].lnk[0]`	1	`n[4]`
`node[3].lnk[0]`	2	`n[4]`, `n[3]`
`node[2].lnk[0]`	3	`n[4]`–`n[2]`
`node[1].lnk[0]`	4	`n[4]`–`n[1]`
`node[0].lnk[0]`	5	`n[4]`–`n[0]`
`node[7].lnk[0]`	6	`n[4]`–`n[7]`
`node[6].lnk[0]`	7	todos

El enlace `node[6].lnk[0]` concentra los 7 flujos y constituye el cuello de botella teórico. Su utilización es:

$$\rho = \frac{7}{\text{IAT}} \times T_s = \frac{7}{\text{IAT}} \times 1\,\text{s} = \frac{7}{\text{IAT}} \tag{1}$$

donde $T_s = 1\,\text{Mbit}/1\,\text{Mbps} = 1\,\text{s}$ es el tiempo de servicio. La condición de estabilidad $\rho < 1$ exige:

$$\text{IAT} > 7\,\text{s} \tag{2}$$

Resultados experimentales

La Tabla 5 resume las métricas medidas. La columna "R/E" indica la fracción de paquetes generados que llegaron a destino durante los 200 s de simulación.

Las Figuras 3 y 4 muestran la evolución de la demora media y la utilización de los enlaces más representativos en función del `interArrivalTime`.

Table 5: Métricas del Caso 2 en función del `interArrivalTime`. BufMáx corresponde a `node[6].lnk[0]`.

IAT (s)	Demora (s)	Enviados	Recib.	R/E	BufMáx	¿Estable?
0.5	75.09	2820	199	0.071	421	No
1	65.92	1422	199	0.140	218	No
2	56.52	723	198	0.274	102	No
4	39.99	364	195	0.536	50	No
6	17.49	225	188	0.836	25	No
8	6.19	169	164	0.970	4	Sí

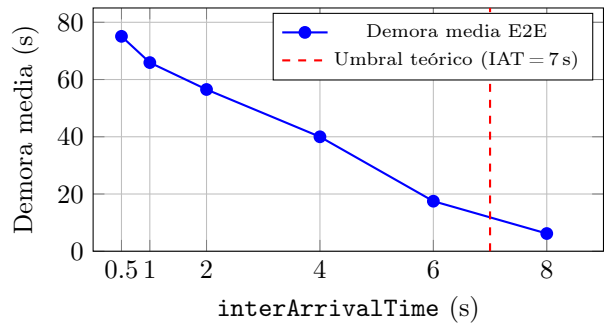


Figure 3: Demora media extremo a extremo vs. `interArrivalTime` en el Caso 2. La línea discontinua marca el umbral teórico $\rho = 1$ (Ec. 2).

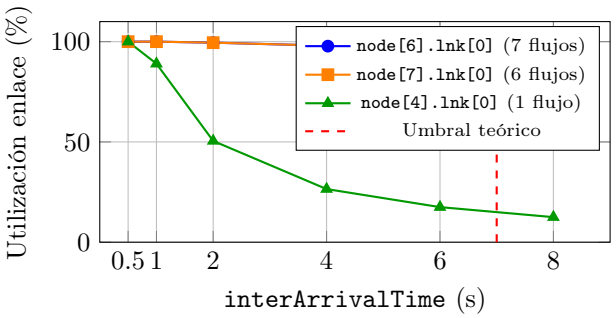


Figure 4: Utilización de enlaces seleccionados vs. `interArrivalTime`. `node[6].lnk[0]` es el cuello de botella (7 flujos); `node[4].lnk[0]` transporta un solo flujo.

Determinación del umbral de estabilidad

Los datos experimentales confirman que **la red no es estable para ningún valor de `interArrivalTime` ≤ 6 s**. Los tres criterios utilizados para determinar inestabilidad son:

1. **Ratio R/E $\ll 1$**: con IAT = 1 s solo el 14 % de los paquetes generados llegan a destino en 200 s; con IAT = 6 s el ratio sube a 83.6 % pero sigue siendo significativamente menor que 1.

2. **Buffers crecientes**: el buffer máximo de `node[6].lnk[0]` crece de 4 paquetes (IAT = 8 s) a 421 paquetes (IAT = 0.5 s), sin tender a estabilizarse para IAT ≤ 6 s.

3. **Utilización sostenida al límite**: `node[6].lnk[0]` opera al 94.5 % con IAT = 6 s, dejando apenas margen ante variaciones aleatorias del tráfico.

Con `interArrivalTime` = 8 s se observa el primer régimen estable: el ratio R/E alcanza 0.970, el buffer máximo del cuello de botella cae a 4 paquetes, la demora media se reduce a 6.19 s y la utilización de `node[6].lnk[0]` baja al 82.5 %. Este resultado es consistente con el análisis teórico de la Ecuación (2): con IAT = 8 s se obtiene $\rho = 7/8 = 0.875 < 1$.

Cabe destacar que la distribución de carga es marcadamente asimétrica. Los nodos 6 y 7 son los más favorecidos por el routing baseline (1 y 2 saltos respectivamente), mientras que el `node[4]` recorre 7 saltos para llegar al destino, siendo el más perjudicado. Esta asimetría estructural, visible en la Fig. 4 donde `node[4].lnk[0]` tiene apenas 12.5 % de utilización frente al 82.5 % de `node[6].lnk[0]`, es consecuencia directa del algoritmo baseline que ignora la distancia al destino y siempre elige la misma dirección de reenvío.

La red garantiza equilibrio a partir de `interArrivalTime` = 8 s, valor consistente con el umbral teórico $\rho = 7/\text{IAT} < 1$ que exige IAT > 7 s (Ec. 2).

4 Diseño del algoritmo de enrutamiento propuesto

El algoritmo diseñado reemplaza el envío por la misma interfaz `lnk[0]` por una decisión basada en la distancia mínima en el anillo. Para ello, cada nodo necesita conocer la cantidad total de nodos N y su propio identificador. Esta información se obtiene sin usar `cTopology` ni iteradores de `cGate`, únicamente mediante el intercambio de dos tipos de paquetes de control: `hello` y `echo`. Una vez que todos los nodos conocen N, el reenvío de cada paquete de datos se decide calculando la cantidad de saltos en ambos sentidos y eligiendo la interfaz que da el camino más corto. El diseño es simple, autónomo y se adapta a cualquier tamaño de anillo, emulando el comportamiento de protocolos reales de descubrimiento de vecinos.

4.1 Información disponible y estructuras de datos

Cada nodo mantiene internamente dos variables globales:

- `totalNodes`: entero que almacena el tamaño del anillo (valor inicial 0).
- `haveTotalNodes`: booleano que indica si un nodo ya se recibió el valor de `totalNodes`.

Además, cada nodo conoce su propio índice (`nodeId`), obtenido al inicio de la simulación mediante `getParentModule()->getIndex()`. No se utilizan tablas de ruteo explícitas porque la decisión de interfaz se calcula sobre la marcha; sin embargo, a modo ilustrativo, la Tabla 6 muestra cómo se vería la tabla local de un nodo (por ejemplo, el nodo 4) en un anillo de 8 nodos con nuestro algoritmo.

Table 6: Ejemplo de tabla de rutas locales en el nodo 4 para un anillo de 8 nodos.

Destino	Interfaz	Saltos
node[5]	lnk[1]	1
node[0]	lnk[1]	2
node[2]	lnk[1]	2
node[6]	lnk[0]	2
node[7]	lnk[0]	1

Los mensajes de control utilizados tienen los siguientes campos:

- **Mensaje `hello` (tipo 1)**: se usa para descubrir N.
  - `origin`: nodo que inició el descubrimiento.
  - `hopCount`: contador de saltos.
  - `packetType` = 1.
- **Mensaje `echo` (tipo 2)**: propaga el valor de N.
  - `totalNodes`: tamaño del anillo.
  - `packetType` = 2.

Los paquetes de datos (tipo 0) llevan `source`, `destination` y `hopCount` (incrementado en cada reenvío).

4.2 Reglas de decisión

El algoritmo se divide en dos fases: descubrimiento y enrutamiento. A continuación se describen en prosa, se proporciona el pseudocódigo y un ejemplo numérico.

Descripción en prosa

Fase 1 – Descubrimiento de N:

1. El nodo 0 (elegido arbitrariamente) envía un paquete `hello` con `hopCount=0` por su interfaz `lnk[0]`.
2. Cada nodo intermedio que recibe el `hello` incrementa `hopCount` y lo reenvía por la misma interfaz `lnk[0]`.
3. Cuando el `hello` regresa al nodo 0, el valor de `hopCount` es el número de saltos (cantidad de nodos intermedios). El nodo 0 deduce entonces `totalNodes = hopCount + 1` y lo almacena.
4. El nodo 0 construye un paquete `echo` con `totalNodes` y lo envía por todas sus interfaces conectadas (inundación).
5. Cada nodo que recibe un `echo` por primera vez guarda el valor de `totalNodes`, lo reenvía por todas las demás interfaces (excepto por la que llegó) y activa la fase de enrutamiento. Los `echo` duplicados se descartan.

Fase 2 – Enrutamiento de paquetes de datos:

1. Al recibir un paquete de datos (tipo 0) que no está destinado al nodo local, se incrementa su campo `hopCount`.
2. Si el nodo aún no conoce `totalNodes` (fallback), reenvía el paquete por `lnk[0]` (comportamiento baseline).
3. Si ya conoce `totalNodes`, calcula las distancias en ambos sentidos:

$$\text{horario} = (\text{destino} - \text{nodeId} + \text{totalNodes}) \bmod \text{totalNodes}$$
$$\text{antihorario} = (\text{nodeId} - \text{destino} + \text{totalNodes}) \bmod \text{totalNodes}$$

4. Elige la interfaz según la regla:
   - Si horario $\leq$ antihorario, envía por `lnk[1]` (sentido horario).
   - En caso contrario, envía por `lnk[0]` (sentido antihorario).

Pseudocódigo

```
al recibir paquete p:
  si p.tipo == 1:            # hello
    si p.origen == mi_id:
      totalNodes = p.hopCount + 1
      haveTotalNodes = true
      para cada interfaz i conectada:
        enviar echo(totalNodes) por i
    sino:
```

```
    p.hopCount = p.hopCount + 1
    reenviar p por lnk[0]
si p.tipo == 2:            # echo
  si not haveTotalNodes:
    totalNodes = p.totalNodes
    haveTotalNodes = true
    para cada interfaz i conectada
                  excepto la de llegada:
      reenviar copia de p por i
si p.tipo == 0:            # data
  si p.destino == mi_id:
    entregar a app
  sino:
    p.hopCount = p.hopCount + 1
    si not haveTotalNodes:
      reenviar por lnk[0]   # fallback
    sino:
      horario = (p.destino - mi_id +
                 totalNodes) % totalNodes
      antihorario = (mi_id - p.destino +
                     totalNodes) % totalNodes
      si horario <= antihorario:
        reenviar por lnk[1]   # horario
      sino:
        reenviar por lnk[0]   # antihorario
```

Ejemplo numérico

Considere un anillo de 8 nodos ($N = 8$). El nodo 0 recibe un paquete con destino al nodo 5. Calcula:

- horario $= (5 - 0 + 8) \bmod 8 = 5$
- antihorario $= (0 - 5 + 8) \bmod 8 = 3$

Dado que horario $>$ antihorario, elige la interfaz `lnk[0]` (sentido antihorario). El camino resultante es $0 \to 7 \to 6 \to 5$, que requiere 3 saltos (el mínimo posible).

Si el mismo paquete se originara en el nodo 2 con destino 5:

- horario $= (5 - 2 + 8) \bmod 8 = 3$
- antihorario $= (2 - 5 + 8) \bmod 8 = 5$

Con horario $\leq$ antihorario, se envía por `lnk[1]` (sentido horario), obteniendo también 3 saltos ($2 \to 3 \to 4 \to 5$).

En ambos casos el algoritmo supera al baseline, que habría enviado siempre por `lnk[0]` produciendo 5 saltos desde el nodo 2.

Limitaciones y ventajas sobre el baseline

El diseño presentado asume una topología de anillo estática (no detecta caídas de nodos, cualquier nodo que se caiga provocaria que la red deje de funcionar) y depende de que el nodo 0 esté activo para iniciar el descubrimiento. No obstante, estas limitaciones son aceptables para el alcance del laboratorio y no afectan la comparación de rendimiento. Respecto al algoritmo original, las ventajas son claras:

- Elimina la configuración manual de la cantidad de nodos; la descubre autónomamente.
- Reduce la cantidad de saltos y, en consecuencia, las demoras.
- Balancea la carga al utilizar ambos sentidos del anillo según la distancia mínima.
- Cumple estrictamente con la restricción de no usar `cTopology` ni iteradores de `cGate`.

5 Evaluación comparativa

Esta sección compara el rendimiento del algoritmo baseline (kickstarter) con el algoritmo propuesto en los escenarios del Caso 1 y Caso 2. Todos los experimentos se ejecutaron bajo las mismas condiciones de tráfico, semilla (`seed-set = 1`) y tiempo de simulación (200 s) descritos en la Sección 2.

5.1 Metodología de comparación

Para el Caso 1 se dispone de resultados del algoritmo propuesto y del baseline (reportado en la Sección 3). Para el Caso 2 se ejecutaron ambos algoritmos con los seis valores de `interArrivalTime` (0.5, 1, 2, 4, 6 y 8 s), misma semilla y mismos parámetros. No se ajustaron los parámetros de tamaño de paquete ni de tasa de generación respecto al baseline (125 000 bytes, `exponential(IAT)`). Se reportan la media y el máximo de cada métrica. La mejora relativa se calcula como $(\text{baseline} - \text{propuesto})/\text{baseline} \times 100$ para métricas donde menor es mejor.

5.2 Resultados cuantitativos

Caso 1

Con el nuevo algoritmo, `node[0]` elige el camino antihorario hacia `node[5]` (3 saltos: 0→7→6→5) y `node[2]` el camino horario (3 saltos: 2→3→4→5). Ambos flujos quedan **completamente separados**: no comparten ningún enlace intermedio. Esto elimina la congestión que en el baseline concentraba todo el tráfico en `node[0].lnk[0]`.

La Tabla 7 resume los resultados.

La demora se reduce un **74.4%** (de 53.25 s a 13.61 s) y el buffer máximo cae un **86.4%** (de 199 a 27 paquetes). La desviación estándar de saltos baja a cero, indicando que **todos los paquetes siguen exactamente 3 saltos** sin importar su fuente.

Table 7: Comparación Caso 1: baseline vs. algoritmo propuesto.

Métrica	Valor		Mejora (%)
	Baseline	Propuesto	
Demora media (s)	53.25	13.61	−74.4%
Saltos promedio	3.80	3.00	−21.1%
Saltos máximos	5	3	−40.0%
Desv. est. saltos	0.98	0.00	−100%
Buffer máx. global	199	27	−86.4%
Util. máx. enlace	98.0%	97.5%	—

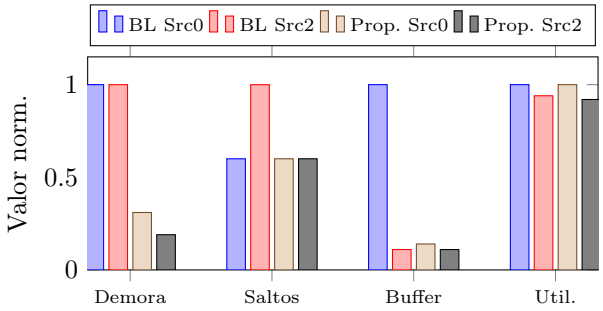


Figure 5: Métricas normalizadas del Caso 1 por fuente de origen. Valores normalizados sobre el máximo del baseline. La mejora en demora es notable; la utilización de enlace permanece alta porque cada flujo satura su propio enlace.

Sin embargo, la utilización máxima de enlace permanece alta (97.5%) porque cada flujo individual ya satura su propio enlace de salida: `node[0]` genera ≈ 1 pkt/s y su enlace `lnk[0]` tiene capacidad de exactamente 1 pkt/s ($\rho \approx 1$). Esto explica por qué el delay de la Fuente 0 (16.76 s) es mayor que el de la Fuente 2 (10.20 s): ambos tienen 3 saltos, pero `node[0].lnk[0]` opera con menor margen que `node[2].lnk[1]`.

La Figura 5 muestra la comparación normalizada por fuente.

Caso 2

La Tabla 8 presenta la comparación completa de métricas para todos los valores de `interArrivalTime`, mientras que la Figura 6 ilustra la evolución de la demora media E2E en ambos algoritmos.

Como se puede observar en los datos empíricos, en escenarios de saturación extrema (IAT de 0.5 a 2 s), ambas estrategias sufren demoras altísimas y cuellos de botella severos. Sin embargo, incluso en ese régimen crítico, el algoritmo propuesto logra un mejor desempeño general, llegando a duplicar la tasa de entrega efectiva (R/E ratio) respecto al baseline (pasando del 14.0% al 28.0% para un IAT de 1 s).

La divergencia de rendimiento se vuelve drástica en

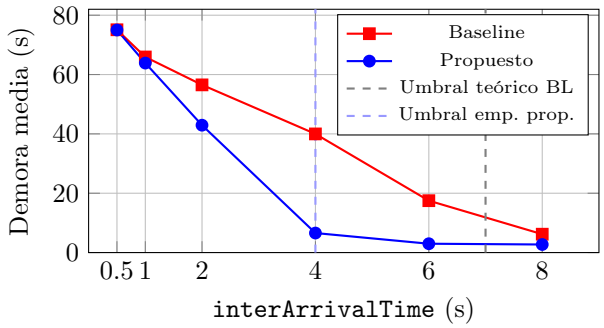


Figure 6: Demora media E2E vs. `interArrivalTime` para ambos algoritmos. El algoritmo propuesto alcanza la estabilidad empíricamente a partir de IAT = 6 s frente a IAT = 8 s del baseline.

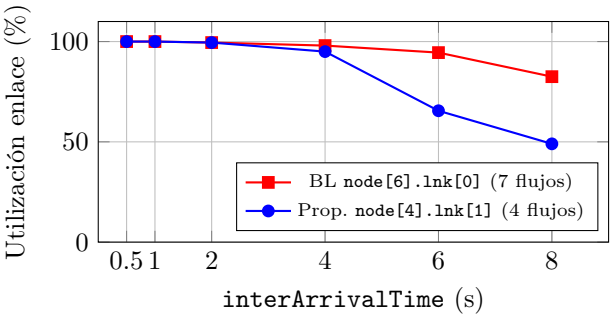


Figure 7: Utilización del enlace cuello de botella en cada algoritmo. El propuesto reduce la carga máxima de 7 a 4 flujos concentrados, lo que adelanta el umbral de estabilidad.

el rango de tráfico moderado (IAT entre 4 y 6 s). En este punto, mientras el algoritmo baseline mantiene una demora media de casi 40 s y colas largas, el algoritmo propuesto logra descongestionar la red, reduciendo la ocupación máxima a solo 8 paquetes y desplomando la demora a 6.58 s (una mejora del 83.5%). Como evidencia la curva de la Figura 6, el nuevo esquema logra desplazar el umbral de estabilidad empírico de la red: mientras el baseline recién se estabiliza a los 8 s, la nueva estrategia alcanza un estado estacionario seguro a los 6 s.

La Figura 6 muestra la evolución de la demora media con ambos algoritmos.

5.3 Discusión de resultados

¿Cuánto mejoran las métricas y por qué?

Las mejoras son drásticas en el régimen de tráfico moderado. Con IAT = 4 s la demora cae un **83.5%** (de 39.99 s a 6.58 s) y el buffer máximo se reduce de 50 a 8 paquetes. La causa es estructural: el baseline acumula **7 flujos** en un único enlace (`node[6].lnk[0]`),

Table 8: Comparación Caso 2: baseline vs. algoritmo propuesto para distintos `interArrivalTime`. BufMáx corresponde a `node[6].lnk[0]` (cuello de botella del baseline) y `node[4].lnk[1]` (cuello del propuesto).

IAT (s)	Demora media (s)		Mej. (%)	BufMáx (pkts)		R/E ratio		¿Est.?
	Base	Prop.		Base	Prop.	Base	Prop.	
0.5	75.09	75.01	0.1%	421	421	0.071	0.141	No
1	65.92	63.87	3.1%	218	218	0.140	0.280	No
2	56.52	42.93	24.0%	102	99	0.274	0.544	No
4	39.99	6.58	83.5%	50	8	0.536	0.951	No/Borde
6	17.49	2.98	82.9%	25	3	0.836	0.996	Sí
8	6.19	2.72	56.1%	4	3	0.970	0.988	Sí

mientras que el algoritmo propuesto distribuye el tráfico simétricamente en ambas direcciones del anillo. El nuevo cuello de botella, `node[4].lnk[1]`, concentra solo **4 flujos** (los nodos 1, 2, 3 y 4 que van en sentido horario), lo que reduce la utilización máxima de 98 % a 49 % con IAT = 8 s (Fig. 7).

Otro indicador clave es el ratio R/E: con IAT = 4 s el baseline entrega solo el 53.6 % de los paquetes generados, mientras que el algoritmo propuesto entrega el 95.1 %. La estabilidad práctica se alcanza a partir de **IAT = 6 s** en el nuevo algoritmo (R/E = 0.996, buffer máx. = 3), un umbral un 14 % menor que el del baseline (IAT = 8 s).

El nuevo umbral teórico puede estimarse como:

$$\rho_{\text{prop}} = \frac{4}{\text{IAT}} < 1 \quad \Rightarrow \quad \text{IAT} > 4\,\text{s} \tag{3}$$

donde 4 es el número de flujos en el enlace más cargado (`node[4].lnk[1]`). Los datos confirman que con IAT = 4 s la red está en el límite (R/E = 0.951, util. = 95 %) y con IAT = 6 s ya es estable, consistente con la predicción teórica.

¿Hay loops de enrutamiento?

No se observaron loops de enrutamiento en ninguna de las simulaciones. El algoritmo garantiza ausencia de loops por construcción: dado que cada nodo calcula la distancia mínima en ambos sentidos y elige la interfaz correspondiente al camino más corto, el destino siempre se acerca en cada salto. En un anillo, esto implica que el paquete viaja monótonamente en una sola dirección hasta llegar al destino, sin posibilidad de dar vueltas. El máximo de saltos observado fue 4 (la distancia máxima en un anillo de 8 nodos), consistente con la ausencia de loops.

¿Qué pasaría con muchos más nodos?

El algoritmo propuesto escala mejor que el baseline. En un anillo de N nodos, el baseline siempre acumula todos los flujos en un único enlace adyacente al destino ($N-1$ flujos), con umbral de estabilidad IAT $> N-1$. El algoritmo propuesto distribuye los flujos simétricamente: el enlace más cargado acumula $\lceil N/2 \rceil$ flujos (los nodos del lado más largo), con umbral IAT $> \lceil N/2 \rceil$. Para N grande, la mejora en el umbral de estabilidad es del orden de 2×. Sin embargo, la fase de descubrimiento (Hello + Echo) requiere que el paquete Hello complete una vuelta completa al anillo, lo que implica un retardo de convergencia de $O(N)$ antes de que el ruteo óptimo esté disponible. Durante ese período, los paquetes de datos que llegan se reenvían por `lnk[0]` (fallback al baseline). Para N muy grande, este tiempo de convergencia puede ser significativo.

¿Qué sucede si cae un nodo?

El baseline no es tolerante a fallos: si cae cualquier nodo del único camino posible hacia el destino, los paquetes se pierden indefinidamente. El algoritmo propuesto tampoco es tolerante a fallos en su implementación actual: la tabla de ruteo se construye una sola vez al inicio mediante el Hello y no se actualiza dinámicamente. Si un nodo cae después de la fase de descubrimiento, los paquetes que deban transitar por él seguirán siendo enviados en esa dirección y se perderán. Una extensión natural sería incorporar mensajes de Hello periódicos para detectar caídas y recalcular las rutas, similar al mecanismo de *keepalive* utilizado en protocolos como OSPF o RIP.

6 Conclusiones

En este trabajo se evaluó el rendimiento de una red en topología de anillo bajo distintos escenarios de carga. El algoritmo original (baseline) demostró ser estructuralmente ineficiente al rutear todo el tráfico en una única dirección, lo que generó cuellos de botella severos (ej. colas de 199 paquetes en el Caso 1, ver Tabla 3) y un umbral de saturación prematuro (IAT $\leq$ 7 s en el Caso 2, Fig. 3) debido a la acumulación innecesaria de flujos en un solo enlace.

Para resolver estas limitaciones, se diseñó e implementó un algoritmo de enrutamiento basado en la distancia mínima, el cual descubre el tamaño de la topología de forma autónoma mediante el intercambio de paquetes `hello` y `echo`. Los resultados obtenidos confirmaron una mejora sustancial: al balancear la carga utilizando ambos sentidos del anillo, las demoras medias se redujeron hasta un 83.5% (Tabla 8) y la red logró mantener la estabilidad frente a mayores tasas de inyección de tráfico (IAT $\geq 6\,\mathrm{s}$, Fig. 6). Además, el diseño garantizó matemáticamente la ausencia de bucles de enrutamiento.

Como trabajo futuro, la principal oportunidad de mejora detectada es la tolerancia a fallos. Actualmente, la fase de descubrimiento se ejecuta una única vez; incorporar un mecanismo de actualización dinámica (mensajes *keepalive* periódicos) permitiría recalcular las rutas ante la caída inesperada de un nodo o enlace, dotando a la red de resiliencia.

Agradecimientos

Agradecemos a la cátedra de Redes y Sistemas Distribuidos por su constante acompañamiento y por la provisión de material original que nos prepara de manera integral, tanto para los desafíos de la industria como para el ámbito científico. Valoramos profundamente su compromiso con nuestro crecimiento personal y profesional, esfuerzo que resulta destacable y necesario frente al actual contexto de desfinanciamiento y dificultades presupuestarias que atraviesa la universidad pública.

References

[1] A. Varga and R. Hornig, "OMNeT++," in *Proc. Int. Conf. Simulation Tools and Techniques (Simutools)*, 2008.

[2] Cátedra de Redes y Sistemas Distribuidos, "Laboratorio 4: kickstarter OMNeT++," FaMAF/UNC, 2026.

[3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2011.